

B.C.A. (Sem. II) Examination, 2026

Session: 2025-28 | Paper Code: 211604

COURSE: 2BCA4 – APPLICATION PROGRAMMING IN FOXPRO

Time: 3:00 Hours	Full Marks: 100	Pass Marks: 40	Format: Question & Solutions
------------------	-----------------	----------------	------------------------------

Note: Candidates are required to give their answers in their own words as far as practicable. The questions are of equal value. Complete, step-by-step solutions for all 10 examination questions are provided below.

QUESTION 1

1. Define a Database Management System and explain the major characteristics of a relational database. Compare the database approach with the traditional file-processing approach.

Answer:

1. Definition of Database Management System (DBMS):

A Database Management System (DBMS) is a software package designed to define, manipulate, store, retrieve, and manage data in a database. It serves as an interface between the database and its end-users or application programs, ensuring that data is consistently organized and remains easily accessible while maintaining high security, integrity, and concurrent access controls.

2. Major Characteristics of a Relational Database System (RDBMS):

- **Tabular Data Structure (Relations):** Data is logically organized into two-dimensional tables consisting of rows (tuples) and columns (attributes).
- **Data Integrity & Constraints:** Supports primary keys, unique constraints, and foreign keys to enforce entity and referential integrity across related tables.
- **Logical & Physical Data Independence:** The internal physical storage mechanism (e.g., indexes, disk files) is isolated from application programs and user queries.
- **Minimization of Redundancy:** Database normalization rules eliminate duplicated data, reducing storage costs and preventing data anomalies.
- **Structured Query Capability:** Provides high-level declarative query languages (such as SQL or FoxPro interactive commands) for efficient data manipulation.

3. Comparison: Database Approach vs. Traditional File-Processing Approach:

Feature / Criterion	Traditional File-Processing Approach	Database Management Approach (DBMS)
---------------------	--------------------------------------	-------------------------------------

Data Redundancy	High. The same data is repeatedly stored in multiple isolated files across departments.	Controlled/Minimal. Data is centralized and normalized to avoid duplicates.
Data Independence	Low. Program code is tightly coupled with physical file structures and formats.	High. Separation of logical data definitions from application programs.
Data Sharing	Difficult. Files belong to individual programs and departments; sharing requires custom scripts.	Easy & Concurrent. Centralized data is shared across multiple users and applications.
Data Integrity	Hard to enforce. Integrity constraints must be manually coded into every application.	Centralized Enforcement. Constraints (Keys, Checks) are defined once in the database schema.
Security & Access	Weak. Relies on simple OS file permissions; lacks fine-grained row/column access.	Robust. Provides granular role-based authorization, encryption, and audit logs.
Backup & Recovery	Manual and complex. Failure during file writes can corrupt the entire dataset.	Automated & Atomic. Transaction logging, rollback, and point-in-time recovery mechanisms.

QUESTION 2

2. Write a FOXPRO program to display the multiplication table of a number entered by the user, from 1 to 10.

Answer:

The following FoxPro program prompts the user to enter any integer number, processes the input, and calculates and displays its multiplication table from 1 to 10 using a FOR...ENDFOR loop.

FoxPro Program: MULTABLE.PRG

```

* Program Name: MULTABLE.PRG
CLEAR
LOCAL nNum, i, nProd

@ 2, 5 SAY "=== MULTIPLICATION TABLE GENERATOR ==="
INPUT "Enter a number: " TO nNum

? ""
? "=====
? "    MULTIPLICATION TABLE OF " + ALLTRIM(STR(nNum))
? "=====

FOR i = 1 TO 10
    nProd = nNum * i
    ? "    " + ALLTRIM(STR(nNum)) + " x " + ;
      PADL(ALLTRIM(STR(i)), 2, " ") + " = " + ;
      ALLTRIM(STR(nProd))
ENDFOR

? "=====
WAIT WINDOW "Multiplication table displayed!" TIMEOUT 3
RETURN

```

QUESTION 3

3. Describe the different data types supported by FoxPro. Give the purpose and typical use of each data type with suitable examples.

Answer:

FoxPro supports a comprehensive set of data types designed for efficient database field definitions and memory variable manipulations. The table and descriptions below detail each data type, its byte size, purpose, and usage examples:

Data Type	Type Code / Size	Purpose & Description	Typical Usage Example
Character	C (1 to 254 bytes)	Stores alphanumeric text strings, names, codes, and addresses.	cName = 'John Doe'
Numeric	N (1 to 20 bytes)	Stores integers or fixed-point decimal numbers for mathematical operations.	nSalary = 45000.50
Float	F (1 to 20 bytes)	Identical storage to Numeric; used for floating-point scientific calculations.	fWeight = 68.75
Date	D (8 bytes)	Stores calendar dates in YYYYMMDD format (displayed via SET DATE).	dDOB = {^2004-05-15}

Logical	L (1 byte)	Stores Boolean true/false values (.T. or .F., .Y. or .N.).	llsActive = .T.
Memo	M (10 bytes in .DBF)	Stores variable-length long text blocks held in associated .FPT memo file.	REMARKS field in DBF
General	G (10 bytes in .DBF)	Stores OLE embedded objects such as images, Word docs, or Excel sheets.	PICTURE field in DBF
Currency	Y (8 bytes)	Stores monetary amounts fixed at 4 decimal places (\$922,337,203,685,477.5807).	yPrice = \$1250.75
Integer	I (4 bytes)	Stores 32-bit signed binary integers (-2,147,483,648 to +2,147,483,647).	nEmpID = 10045
DateTime	T (8 bytes)	Stores combined date and time stamps (MM/DD/YY HH:MM:SS AM/PM).	tLogin = DATETIME()

QUESTION 4

4. Explain the use of conditional and looping statements in FoxPro. Illustrate IF....ELSE, DO WHILE and FOR...ENDFOR with suitable examples.

Answer:

Control structures direct the execution flow of a FoxPro program. Conditional statements execute specific blocks of code based on dynamic logical conditions, while looping statements execute a block of commands repeatedly until a specified condition is satisfied.

1. Conditional Statement: IF...ELSE...ENDIF

Executes one set of instructions if a logical expression evaluates to .T., and an optional alternative set if it evaluates to .F..

Conditional Example: IF...ELSE...ENDIF

```
* Example of IF...ELSE...ENDIF
LOCAL nMarks
INPUT "Enter student marks (0-100): " TO nMarks

IF nMarks >= 40
    ? "Result: PASSED"
    ? "Congratulations! You qualified the examination."
ELSE
    ? "Result: FAILED"
    ? "Please re-appear in the supplementary exam."
ENDIF
```

2. Looping Statement: DO WHILE...ENDDO

A pre-test loop that repeatedly executes the body statements as long as the conditional expression remains true (.T.).

Looping Example: DO WHILE...ENDDO

```
* Example of DO WHILE...ENDDO
LOCAL nSum, nCount
nSum = 0
nCount = 1

DO WHILE nCount <= 5
    nSum = nSum + nCount
    ? "Count: " + STR(nCount) + " | Sum: " + STR(nSum)
    nCount = nCount + 1
ENDDO

? "Final Cumulative Sum (1 to 5): ", nSum
```

3. Counter-Controlled Loop: FOR...ENDFOR

Executes a block of commands a fixed number of times using an explicit loop counter variable with an optional step increment.

Looping Example: FOR...ENDFOR

```
* Example of FOR...ENDFOR
LOCAL i, nSquare

? "Number | Square"
? "-----"
FOR i = 1 TO 10 STEP 2    && Increment by 2
    nSquare = i * i
    ? "    " + STR(i, 2) + " |    " + STR(nSquare, 3)
ENDFOR
```

QUESTION 5

5. Develop a FoxPro program that accepts 10 numbers into an array, identifies the largest and smallest values, and displays their difference.

Answer:

The program below declares a 1-dimensional array of size 10 using DIMENSION, populates it with user inputs, scans the array to determine the minimum and maximum values, and displays the net difference.

FoxPro Program: ARRAYMINMAX.PRG

```
* Program Name: ARRAYMINMAX.PRG
CLEAR
DIMENSION aNumbers(10)
LOCAL i, nMax, nMin, nDiff, nVal

? "=== ARRAY PROCESSING - 10 NUMBERS ==="
FOR i = 1 TO 10
    INPUT "Enter element " + ALLTRIM(STR(i)) + " of 10: " TO nVal
    aNumbers(i) = nVal
ENDFOR

* Initialize Max and Min with first array element
nMax = aNumbers(1)
nMin = aNumbers(1)

* Traverse array to find Max and Min
FOR i = 2 TO 10
    IF aNumbers(i) > nMax
        nMax = aNumbers(i)
    ENDIF
    IF aNumbers(i) < nMin
        nMin = aNumbers(i)
    ENDIF
ENDFOR

nDiff = nMax - nMin

? ""
? "=====
? "   Largest Value (Max) : " + ALLTRIM(STR(nMax))
? "   Smallest Value (Min) : " + ALLTRIM(STR(nMin))
? "   Difference (Max - Min): " + ALLTRIM(STR(nDiff))
? "=====

RETURN
```

QUESTION 6

6. Explain the following FoxPro concepts:

- (i) User-defined Function**
- (ii) Parameter Passing**
- (iii) Index File**
- (iv) Record Sorting**

Answer:

- (i) User-Defined Function (UDF):**

A User-Defined Function (UDF) is a custom routine written by the programmer to perform specific computational tasks that are not built into FoxPro. UDFs begin with a FUNCTION statement, end with ENDFUNC, and return a value using the RETURN clause. They can be called directly within expressions, command outputs (e.g., ? CalculateTax(salary)), or database queries.

(ii) Parameter Passing:

Parameter passing is the mechanism by which data values are transferred into functions or procedures. FoxPro supports two parameter passing techniques:

- **Pass by Reference (Default for DO...WITH):** The memory address of the variable is passed. Any changes made inside the routine directly alter the original variable in the calling program.
- **Pass by Value (Default for Functions UDFs):** A copy of the value is passed. Enclosing variables in parentheses (e.g., MyFunc((x))) forces pass-by-value, protecting original variables from modification.

(iii) Index File:

An index file is an auxiliary file (.IDX single-index or .CDX compound-index) containing pointers ordered logically according to a designated key expression. Indexing does not physically alter the position of records in the underlying .DBF table file; instead, it maintains an ordered tree index for lightning-fast record searches (via SEEK or FIND commands) and ordered view outputs.

(iv) Record Sorting:

Record sorting is the physical reorganization of database records. Executing the SORT TO <newfile> ON <field> command physically creates a brand-new .DBF file on disk with all rows arranged in ascending or descending sequence. Unlike indexing, sorting requires substantial extra disk storage and creates a static duplicate copy that does not automatically update when original records change.

QUESTION 7

7. Distinguish between a procedure, a function and a command in FoxPro. Write a user-defined function to determine whether a given number is prime or not.

Answer:

1. Distinction Between Procedure, Function, and Command:

Aspect / Feature	FoxPro Command	Procedure (PROCEDURE)	User-Defined Function (FUNCTION)
Definition	Built-in language keyword executed directly by the runtime engine.	Subprogram designed to perform a series of sequential tasks.	Subprogram designed primarily to compute and return a value.
Invocation Syntax	Executed directly (e.g., USE student).	Invoked via DO <procname> [WITH ...].	Invoked inline within expressions (e.g., x = IsPrime(n)).
Return Value	Never returns a value directly.	Can return values, but optional.	Must return a value using RETURN <expr>.

Usage Context	Standalone program statement.	Standalone routine block.	Embedded in ?, assignment statements, or REPLACE.
---------------	-------------------------------	---------------------------	---

2. User-Defined Function (UDF) for Prime Number Testing:

FoxPro UDF: ISPRIME.PRG & Test Driver

```
* Function: ISPRIME.PRG
FUNCTION IsPrime
    LPARAMETERS nNum
    LOCAL i, lPrimeFlag

    IF nNum < 2
        RETURN .F.
    ENDIF

    lPrimeFlag = .T.
    FOR i = 2 TO INT(SQRT(nNum))
        IF MOD(nNum, i) == 0
            lPrimeFlag = .F.
            EXIT
        ENDIF
    ENDFOR

    RETURN lPrimeFlag
ENDFUNC

* --- TEST DRIVER PROGRAM ---
CLEAR
LOCAL nTest
INPUT "Enter an integer to test for Primality: " TO nTest

IF IsPrime(nTest)
    ? "Result: " + ALLTRIM(STR(nTest)) + " is a PRIME number."
ELSE
    ? "Result: " + ALLTRIM(STR(nTest)) + " is NOT a prime number."
ENDIF
```

QUESTION 8

8. Write the syntax and explain the purpose of the following FoxPro Interface commands:

- (i) DEFINE WINDOW
- (ii) DEFINE MENU
- (iii) DEFINE POPUP
- (iv) @...SAY/GET commands

Answer:

- (i) DEFINE WINDOW:

- **Purpose:** Creates a custom user-interface window definition specifying dimensions, title, border styles, and screen position prior to activation.

- **Syntax:** DEFINE WINDOW <WinName> FROM <r1,c1> TO <r2,c2> [TITLE <cTitle>] [SYSTEM|DOUBLE|NONE] [COLOR SCHEME <n>]

- **Example:** DEFINE WINDOW wEntry FROM 5, 10 TO 20, 70 TITLE 'Data Entry' DOUBLE SYSTEM

(ii) DEFINE MENU:

- **Purpose:** Defines a horizontal menu bar container at the top of the screen or window.

- **Syntax:** DEFINE MENU <MenuName> [BAR] [KEY <KeyLabel>] [COLOR SCHEME <n>]

- **Example:** DEFINE MENU mMain BAR COLOR SCHEME 3

(iii) DEFINE POPUP:

- **Purpose:** Defines a vertical drop-down or pop-up list menu containing individual selectable options (bars).

- **Syntax:** DEFINE POPUP <PopupName> FROM <r1,c1> [TO <r2,c2>] [PROMPT FIELD <Field>] [TITLE <cTitle>]

- **Example:** DEFINE POPUP pFile FROM 2, 10 PROMPT BAR TITLE 'File Options'

(iv) @...SAY/GET Commands:

- **Purpose:** Formats screen output ('SAY') and interactive data input fields ('GET') at exact row and column coordinates. The input fields are activated collectively when a READ command is issued.

- **Syntax:**

```
@ <row>, <col> SAY <expC> [PICTURE <mask>]
```

```
@ <row>, <col> GET <memvar|field> [PICTURE <mask>] [VALID <!Exp>]
```

```
READ
```

- **Example:**

```
CLEAR
```

```
cName = SPACE(20)
```

```
@ 5, 10 SAY 'Enter Student Name:' GET cName PICTURE '!' VALID !EMPTY(cName)
```

```
READ
```

QUESTION 9

9. Write short notes on any two of the following:

(i) Database File and Table

(ii) Fields, Records and Memo Fields

(iii) Index Expressions and Tags

(iv) Reports and Labels

Answer: (Detailed notes for all 4 sub-topics are provided below):

(i) Database File and Table:

In FoxPro, a **Table (.DBF file)** is a structured data file containing rows and columns storing raw data records. A **Database Container (.DBC file)** acts as a logical master wrapper that organizes multiple .DBF tables, defining persistent relationships, input field validation rules, stored procedures, and triggers across the tables.

(ii) Fields, Records, and Memo Fields:

- **Fields:** Columns in a table defining distinct attributes (e.g., Name, Age) with specified data types and fixed widths.
- **Records:** Horizontal rows representing individual data entities stored sequentially.
- **Memo Fields:** Variable-length text fields. The .DBF table holds only a 10-byte pointer referencing the actual text block stored in an associated .FPT memo file, conserving table space.

(iii) Index Expressions and Tags:

An **Index Expression** is a combination of FoxPro functions and field names (e.g., UPPER(lname) + UPPER(fname)) used to compute index keys. A **Tag** is an individual named index key stored inside a structural Compound Index File (.CDX). Multiple tags can exist inside a single .CDX file, permitting instant switching of table sort orders via SET ORDER TO TAG <TagName>.

(iv) Reports and Labels:

- **Reports:** Printable document layouts generated via FoxPro Report Designer (CREATE REPORT) and executed using REPORT FORM <name> PREVIEW|TO PRINTER. Structured into Title, Page Header, Detail, Page Footer, and Summary bands.
- **Labels:** Standardized mailing label or badge layouts formatted via CREATE LABEL and printed using LABEL FORM <name> PREVIEW.

QUESTION 10

10. Explain the Fox Pro commands used to display and manipulate graphical elements such as boxes and lines. Write suitable command examples for each.

Answer:

FoxPro provides text-mode character graphics commands designed to draw box borders, horizontal/vertical separator lines, and clear specific screen rectangular areas for polished user interface forms:

1. Screen Box Drawing Command: @ <r1,c1> TO <r2,c2>

Draws a rectangular box boundary between top-left coordinate (r1,c1) and bottom-right coordinate (r2,c2). Clauses include DOUBLE (double line border), PANEL (shaded border), or custom string border patterns.

2. Custom Box Command: @ <r1,c1> BOX <cString>

Draws a box using a specified 8-character string defining custom characters for corners, top/bottom edges, and side walls.

3. Line Drawing Command:

Executed using @ r1, c1 TO r2, c2 where setting r1 = r2 draws a horizontal line, and setting c1 = c2 draws a vertical line.

4. Area Clearing Command: @ r1, c1 CLEAR TO r2, c2

Clears text and graphic lines within a specified rectangular coordinate region.

FoxPro Screen Layout & Graphics Demo

```
* Program Name: GRAPHICS_DEMO.PRG
CLEAR

* 1. Draw double-line outer border box
@ 2, 5 TO 20, 75 DOUBLE

* 2. Draw panel header box
@ 3, 7 TO 5, 73 PANEL
@ 4, 22 SAY "ACADEMIC SYSTEM USER INTERFACE DEMO"

* 3. Draw horizontal separator line
@ 6, 6 TO 6, 74

* 4. Partition screen with vertical line
@ 7, 40 TO 18, 40

* 5. Display text in partitioned columns
@ 8, 10 SAY "=== LEFT COLUMN ==="
@ 10, 10 SAY "1. Student Registration"
@ 12, 10 SAY "2. Marks Entry System"

@ 8, 45 SAY "=== RIGHT COLUMN ==="
@ 10, 45 SAY "3. Examination Results"
@ 12, 45 SAY "4. Grade Card Printing"

* 6. Draw custom box using string characters
@ 15, 10 BOX "+---+" + "| |" + "+---+"
@ 16, 17 SAY "Custom Box Border Example"

WAIT WINDOW "Press any key to close screen demo..."
RETURN
```